Runspindle: Authorized Work as a Verifiable System

November 2026 | Design scaffold

No live implementation, deployment, benchmark, or savings claim.

A bounded system for authorized work

Runspindle is a proposed distribution of cooperating components for agent-assisted work. Its aim is to make task boundaries, human control, evidence, and recovery explicit. The current deliverable is a scaffold and design specification. It is not an operating service, a production deployment, or a measured improvement over a native agent product.

A task begins with a defined outcome, accountable owner, permitted sources and actions, and stop conditions. An agent's ability to perform an action is not authority to perform it. Scope changes and consequential actions require the applicable approval. Product policy and legal constraints remain in force; the architecture is not a mechanism for bypassing them.

Why a specialist harness might help

A general-purpose agent can leave work unfinished even when a bounded, permitted path exists. The useful engineering question is whether a narrow integration can improve verified completion without expanding authority. Additional prompting, retries, and tool calls can also increase cost or create duplicate external effects. Runspindle therefore treats completion, authorization, and recovery as separate things to verify.

A future comparison should hold the task and permission boundary constant. Native assistance, expert-guided assistance, and the proposed harness should receive matched tasks. Count failures, human intervention, and uncertain results, rather than reporting only successful examples. No such comparative Runspindle study has been completed.

Seven components with separate responsibilities

Core is planned in Rust with Temporal for durable task state, approval waits, and explicit retry and reconciliation decisions. Host is the proposed native Codex integration and owns host/session identity checks. Browser is planned around OpenBao secret references, PinchTab actions, and a KasmVNC human view of the same browser.

Tether is intended to adapt the existing Tether Slack integration, with verified actors and bounded routing rather than granting authority from message arrival. Refinery is the proposed Python DSPy experiment layer. Console is intended to show actual fleet observations, including their age and unknown states; the current scaffold has no connected fleet. Platform owns the proposed distribution manifests, release packaging, development environment, and deployment gates.

These are component contracts, not assertions that their integrations work. Naming a technology does not establish compatibility, availability, or a security guarantee.

Browser control and human takeover

The intended browser model is one isolated account context, one headed browser, and one current controller. A broker would bind commands to account, task, actor, and lease epoch. PinchTab would provide the action interface. KasmVNC passive viewing is proposed as a separate, authorized

read-only mode that grants no input control. Interactive takeover would require a fresh fenced control lease on the same browser. A handoff must invalidate old commands and revoke existing input and streams before granting a new epoch.

OpenBao is proposed for secret references and controlled credential access. Credentials and browser profiles must not enter logs, public artifacts, or model evaluation datasets. Temporal records durable intent; it cannot by itself fence browser input or prove whether a website accepted a submission. After an ambiguous external action, the workflow must reconcile observable state before retrying. These properties still require Runspindle integration and adversarial tests.

Refinement without default collection

The proposed refinery begins only with an explicitly selected workflow and authorized examples. Review and redaction precede dataset entry. Dataset versions should retain source permission, adjudication, group identifiers, and stable train, validation, and held-out splits. Related examples must remain in the same split to reduce leakage. Proposed feedback records would bind attributable reactions or replies to the exact output and its version. Task-specific automatic gold rules may admit examples only under explicit criteria; conflicting signals must be quarantined for adjudication. Gold eligibility is separate from candidate release approval.

Python DSPy candidates would be compared with a baseline under a fixed task definition and evaluation budget. Quality, safety failures, latency, human repair, and total cost belong in the comparison. Experiments cannot promote themselves. A versioned release needs threshold evidence, human authorization, and a rollback reference. No live collection, provider invocation, training, promotion, or saving is claimed.

Economics of accepted work

Cost per accepted authorized task is total model, tool, human, verification, maintenance, and recovery cost divided by the number of verified authorized completions. Include failed attempts and cleanup in the numerator. If there are no accepted completions, report the cost and zero completions rather than a misleading unit price.

A specialist system also has fixed integration and evaluation costs. Low-volume workflows may remain cheaper with manual assistance. Break-even depends on measured workload mix, acceptance rate, maintenance cadence, and labor assumptions. Saved minutes do not automatically become cash savings. Provider improvements may remove the advantage of a custom harness, so repeat comparisons after material model or platform changes. Runspindle has no measured savings, prices, break-even point, or market-size estimate.

Fleet evidence and uncertainty

A fleet view should distinguish a planned component, an observed process, a managed session, and a verified terminal result. Every observation needs a timestamp, source, and freshness policy. Missing updates mean unknown state, not automatic proof of failure. Control actions need a verified binding to the intended host, repository, process, and session.

Console should eventually render those actual observations rather than synthetic success indicators. Tether messages may link to a bounded task or evidence record, but message delivery is not execution completion. A stale route, recycled process identifier, or disconnected host must not silently redirect an action. Standard Collectors and Langfuse integration are proposed for correlated task, session, trace, and output identity with readback verification. Masking coverage remains unresolved across those

paths; no complete redaction guarantee is claimed. The scaffold does not yet collect or render live observations.

Evaluation and release gates

Development validation should test contracts and synthetic fixtures first: lease fencing, isolation, duplicate requests, restart recovery, permission changes, cancellation, stream revocation, and unknown external outcomes. Integration tests then need pinned component versions and permissioned environments. No source repository's historical test result transfers automatically to the new system.

Production readiness requires independently evidenced secrets management, authenticated routing, monitoring, rollback, dependency review, and operator acceptance. Packaging a manifest is not deployment; a passing stub test is not proof of a live system. Development and production deployment remain planned. The distribution must fail closed when prerequisites or authorization are missing.

Claims, sources, and open questions

This migration adapts selected concepts under opaque source identifiers SRC-002, SRC-013, and SRC-009. These are partial migrations and newly written adaptations, not faithful re-renders. Historical PDFs are preserved separately and are not Runspindle builds. Detailed source provenance and review records are kept outside the public candidate. Source selection does not establish a new project license or third-party endorsement.

The next useful evidence is a reproducible, permissioned end-to-end task that traverses the component boundaries and demonstrates recovery from an uncertain result. Until then, the materials describe a research and engineering plan. Open questions include native session integration, browser lifecycle compatibility, least-privilege secret delivery, evaluation validity, release reproducibility, and operational cost.